# Terminal Interface for the Arduino and Raspberry Pi v1.0

© 2026 MicroPhonon LLC.  Licensed under Creative Commons



This software provides a text-only terminal interface for two-way communication between the AquaPing acoustic leak sensor and a personal computer. Sensor data is read and displayed; its monitoring parameters can also be configured. Because the AquaPing sends and receives data bytes using the I2C protocol, intermediary hardware is required when using a laptop or desktop computer. Widely available and inexpensive Arduino or Raspberry Pi microcontrollers can perform the function of I2C master.  The master also relays sensor/slave data to and from a PC. Because of their emphasis on low-power IOT applications using a 3V3 power bus, the code has been developed for two specific hardware types:

- Arduino Nano 33 IOT. Load the arduino_terminal.ino firmware
- Raspberry Pi Pico. Load the raspi_terminal.ino firmware

Minor modifications are needed to implement wireless Bluetooth or WiFi as well as other Arduino and Raspberry Pi variants.  Appropriate edits can be determined by a competent LLM.

Also found in this directory is some older code written for the Lolin D1-mini WiFi board:

- D1_mini.ino. The D1 mini transfers data between PC and sensor
- ThingSpeak.ino. D1 mini is used to: i) transfer data between PC and sensor and ii) upload data to ThingSpeak server over WiFi

The software is open-source and licensed under Creative Commons. Refer to the user manual for detailed description and interpretation of the sensor data stream and configuration of the control parameters. A graphical user interface is also available, but different firmware must be loaded.

**Installation**. The free Arduino IDE is used to flash the .ino firmware onto the device via USB serial. The Raspberry Pi Pico uses the Earle Philhower Raspberry Pi Pico/RP2040 board package. It can be installed by following these instructions:

https://learn.adafruit.com/using-raspberry-pi-pico-pico-2-with-arduino/rp2040-arduino-ide-setup

The two files arduino_terminal.ino and raspi_terminal.ino differ primarily by the pin assignments.

**Hardware**. The AquaPing is designed to run from a *quiet* DC power source in the range 1.8–3.3V. Suitable supply voltage can be obtained directly from the Arduino Nano 33 IOT and Raspberry Pi Pico; consult the board schematic for the designated +3V3 and GND pins. The default power-save-mode of the RP Pico, however, produces excessive ripple on the 3V3 line that will send the front-end amplifier stages of the AquaPing into oscillation. The raspi_terminal.ino firmware switches it into a quieter PWM mode. In both cases, placing a large electrolytic decoupling capacitor across the 3V3 power bus is mandatory. The AquaPing will be damaged if it is exposed to 5 Vdc and higher. If only 5V is available, an inexpensive voltage regulator or step-down converter must be used.

Caution: The Nano 33 IOT is known to generate noise transients on the power bus when configured for wireless operation, i.e. during the brief intervals when the radio is transmitting. Additional precautions may be needed to prevent disruption of AquaPing operation.

A USB Micro-B cable connects the PC to the I2C master hardware at a baud rate of 9600. Jumper wires link the master to corresponding I2C ports on the AquaPing. On the Nano 33 IOT, the SDA and SDL pins are A4 and A5, respectively. The RP Pico uses GP4 and GP5, respectively. Pullup resistors are already present on the Nano 33 IOT, but the RP Pico does not have this feature. External pullup resistor values of 4.7k (not critical) are a good choice for a clock speed of 100 kHz; the pullup voltage is also 3V3. Use of a higher pullup voltage will likely damage the AquaPing.

Immediate alerts (i.e. without waiting a full polling period to perform an I2C read) are possible by wiring the ALARM and/or NOISE pins on the AquaPing to interrupt ports as configured on the master. This can be useful when the master is set for very long delay between reads. The latency is then determined by the environmental polling period of the AquaPing. The interrupt lines also allow the sensor to be optionally deployed without wiring the I2C interface. The .ino file for the Nano 33 IOT is coded for interrupts on pins D3 and D2 for ALARM and NOISE, respectively. The RP Pico firmware sets the interrupts on GP3 and GP2, respectively. Modify as needed.

**Operation:** On initial power up, the AquaPing will immediately enter Training Mode as described in the user manual. The master device will begin retrieving data from it via I2C at a 1 Hz rate. The LED located at the corner of the Arduino/RP blinks to signal each transfer. The master polling rate is adjusted by changing the polling_interval constant in the .ino firmware; default is 5000 (ms).

It is important to understand that the master and slave (AquaPing) are operating independently and asynchronously. Acoustic data transport from the ADC along with I2C read/write operations are handled without CPU intervention by Direct Memory Access (DMA) on the MSP430 microcontroller. The acoustic monitoring interval of the AquaPing and the polled I2C read events are entirely separate operations and user-configurable.

A UART serial terminal program like putty or minicom can be used to monitor sensor operation and send it a variety of commands. The user must specify the proper serial port. The On Linux, this may look like /dev/ttyUSB0 or dev/ttyACM0; they will be shown as COM ports on Windows. The preset baud rate is 9600. When the terminal program connects to the master hardware, a 12-number data stream is continuously read from the AquaPing and displayed in decimal format. Leak and Noise alerts are also sent to the terminal.

The following eight commands are converted to byte strings by the software and passed to the sensor via I2C. The structure of all commands except reset (single character r) is a single lower-case alphabetic, followed by a 1–3 digit numeric, and the Enter key. As an example, the polling period is set to 10 seconds by entering the command: o5

Multiple commands can be sent provided there is a single space between them. Commands can be sent in any order as long as the proper syntax is used. Example: The data collection set is resized to 128 and LEDs are turned off by entering: e128 l0

The commands shown in Table I create persistent settings that remain in the sensor’s non-volatile memory after power cycling:

| Parameter | Lower case character | Numeric arguments | Corresponding values |
|---|---|---|---|
| LEDs | l | 0 or 1 | On or Off |
| Background | b | 10–255 | 10–255 |
| Polling period | o | 1–9 | 1, 2, 3, 5, 10, 15, 20, 25, 30 sec |
| Event array size | e | 10--255 | 10–255 |
| Trigger count | t | 10–255 | 10--255 |

Table I. Persistent sensor parameters

The commands shown in Table II do not affect persistent memory. The sensor is always in an active state after power cycling.

| Control | Lower case character | Arguments | Action |
|---|---|---|---|
| Power state | p | 0 or 1 | Sleep or Active |
| Alarm | a | 0 or 1 | Enable or Disable |
| Reset | r | | Set all parameters to defaults |

Table II. Non-persistent sensor controls

All of the above commands with the exception of the LED toggle will erase any accumulated data and reset the counting arrays. The sensor will reject a setting with Trigger count > Event array size.